22 基礎圖演算法

Part VI Graph Algorithms

本章定位

圖的搜尋順序同時決定距離、遞迴祖先與完成時間；本章用同一六點有向圖，從FIFO層序與白路徑定理推導BFS、DFS、拓撲序及SCC的正確性與線性成本。

前置知識

圖表示、顏色、時間戳與線性走訪。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 選擇 adjacency matrix 或 list。
- 推導 BFS 最短路徑性質。
- 用 DFS 完成 topological sort 與 SCC。
- 22.1 Representations of graphs
- 22.2 Breadth-first search
- 22.3 Depth-first search
- 22.4 Topological sort
- 22.5 Strongly connected components

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

22.1 Representations of graphs

本章的圖是有限有向圖或無向圖；每個 vertex（頂點）有穩定識別碼，edge（邊）的方向必須明定。路徑可以長度零，因此每個頂點可達自身。令 $n=|V|$、$m=|E|$；無向邊只在 E 計一次。除特別註明，本章的最短距離是**邊數**，不是任意權重的總和。

Adjacency list（鄰接串列）有 n 個表頭。有向圖的所有表長之和為 m；無向簡單圖每條邊在兩端各存一次，總長為 2m。空間 $\Theta(n+m)$，列出 u 的鄰點需 $\Theta(1+\text{outdegree}(u))$，掃描普通未排序串列查詢 (u,v) 最壞 $\Theta(1+\text{outdegree}(u))$。雜湊或排序的鄰接集合可以改變查詢成本，不能把它們的保證套到普通 list。

Adjacency matrix（鄰接矩陣）A 的 row 是起點、column 是終點，$A[u,v]=1$ 表示 $u \to v$；空間 $\Theta(n^2)$、查邊 $\Theta(1)$，但枚舉 u 的鄰點一般要掃 n 個位置。無向圖滿足 $A=A^{\mathsf{T}}$。有權矩陣需另有「不存在」標記：權重 0 可以是合法邊，不能直接與不存在混為一談。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

同一個六頂點有向圖

後面的 BFS、DFS 與 SCC 都使用下列**同一組有向邊**，鄰接與外層頂點順序均按表排列。

頂點	Adj[u]	outdegree	indegree
a	b,c	2	1
b	c	1	2
c	a	1	2
d	b,e	2	1
e	d	1	2
f	e	1	0

n=6、m=8，表長總和 8，入度總和也為 8；不是 16。轉置圖 G^T 把每個 u→v 放入 $Adj^T[v]$ 的 u 項：先建立 n 個空表，再掃全部 m 項，時間與新增空間皆 Θ(n+m)。原圖不需破壞。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

同一個六頂點有向圖 （續）

一張圖的 List 與 Matrix

Adj[u]：row u → column v；這是有向圖，m=8。

頂點	鄰接串列
a	b,c
b	c
c	a
d	b,e
e	d
f	e

A 的 column

	a	b	c	d	e	f
a	0	1	1	0	0	0
b	0	0	1	0	0	0
c	1	0	0	0	0	0
d	0	1	0	0	1	0
e	0	0	0	1	0	0
f	0	0	0	0	1	0

綠格：存在的有向邊；8個綠格與8個list項逐一對應。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

22.2 Breadth-first search

BFS（廣度優先搜尋）以 FIFO queue 維持距離的層序。白色是未發現，灰色是已發現而尚未掃完，黑色是鄰接表已掃完。π[v] 是發現 v 的前驅；∞ 與 NIL 分別表示尚無距離與無前驅，不能用 0 取代。

```
BFS(G,s):
  for v in V: color[v]=WHITE; d[v]=∞; π[v]=NIL
  color[s]=GRAY; d[s]=0; Q=empty queue; enqueue(Q,s)
  while Q is not empty:
    u=dequeue(Q)
    for v in Adj[u]:
      if color[v]==WHITE:
        color[v]=GRAY; d[v]=d[u]+1; π[v]=u
        enqueue(Q,v)
    color[u]=BLACK
  return d,π
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

入列之前就改色，同一頂點才不會因兩個父候選重複入列。在每輪 while 開始時，queue 恰為全部灰色頂點；掃描 u 的期間，u 已出列但仍灰色，因此不要把這個不變量誤稱為每一條指令後都成立。

FIFO 為何保證先處理較近的點

把 queue 依隊首到隊尾寫為 $q_1,\cdots,q_r$。其距離非遞減，且 $d[q_r] \le d[q_1]+1$；空 queue 時條件視為成立。

初始只有 s。若本輪取出 u、$d[u]=k$，剩餘項依歸納只有 k 或 k+1，且非遞減；新發現的每個 v 都取得 k+1 並接在尾端，因此順序保持。本輪結束若 k 層已清空，隊首變成 k+1，下一輪才可能產生 k+2。由此每次 dequeue 的距離也非遞減。

每個有限 d[v] 都有一條實際 π 路徑：s 的長度 0；發現 v 時把 u 的路徑接 u→v，長度恰 $d[u]+1$。因此真最短距離 $\delta(s,v) \le d[v]$，而 π 鏈的 d 每次下降 1，不可能形成 cycle。

但「找到一條路」本身只給上界，還不足以證明最短；下一步需要排除先找到較長路徑的可能。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

BFS 最短距離的完整論證

假設有可達 v 最後 $d[v] \neq \delta(s,v)$，選 δ 最小的這種 v。$v \neq s$；取最短路徑中 v 的前一點 u，則 $\delta(s,u)=\delta(s,v)-1$，且依選法 $d[u]=\delta(s,u)$。u 有限距離，必定入列並最終出列。

掃描 $u \to v$ 時，若 v 仍白，就設 $d[v]=d[u]+1=\delta(s,v)$，矛盾。若 v 已發現，設它最初由 w 掃描發現。w 不晚於 u 出列；由 queue 層序，$d[w] \leq d[u]$，所以 $d[v]=d[w]+1 \leq d[u]+1=\delta(s,v)$。結合上一頁 $\delta \leq d$，仍得到相等。兩種情況都矛盾，故所有可達點的 d 均正確；不可達點若曾被發現就有實際路徑，亦矛盾，故其 d 保持 ∞。

每個可達非根點有一個 π，反向回溯恰 d[v] 條邊後到 s；它們構成 BFS tree。鄰接順序可以改變 π 與最短路徑的選擇，但不能改變 d。

初始化 $\Theta(n)$，每個頂點最多入列與出列一次，每個可達點的鄰接表掃一次，總時間 $O(n+m)$，queue 與 d、π、color 的額外空間 $O(n)$。使用 matrix 時，每個已出列頂點掃 n 格，最壞 $\Theta(n^2)$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

從 d 出發的完整 Queue Trace

表中的 queue 是本輪**掃描結束後**的內容；括號內是 d 值，新增點同時取得前驅。

取出的 u	掃描結果	輪末 queue
尚未開始	初始化 d[d]=0	d(0)
d	b 白→d[b]=1,π[b]=d；e 白→d[e]=1,π[e]=d	b(1),e(1)
b	c 白→d[c]=2,π[c]=b	e(1),c(2)
e	d 已黑，不再入列	c(2)
c	a 白→d[a]=3,π[a]=c	a(3)
a	b、c 都已發現，不變	空

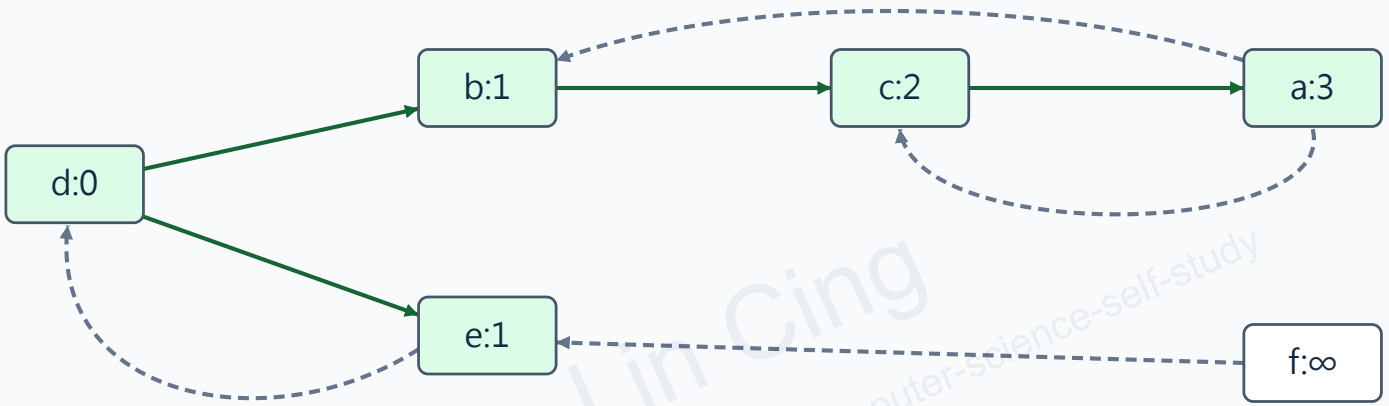
最後 (d[a],d[b],d[c],d[d],d[e],d[f])=(3,1,2,0,1,∞)，到 a 的最短路徑是 d→b→c→a。f→e 的存在不表示 d 可逆向到 f。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

從 d 出發的完整 Queue Trace （續）

BFS：同一六點圖從 d 出發

格內 = 頂點:距離；綠實線 = π邊；灰虛線 = 其他原圖邊。



隊首順序 d(0) → b(1) → e(1) → c(2) → a(3)；f 保持∞。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

有向邊距離、等權前提與最短路徑計數

對任一來源可達的 u→v，δ(s,v)≤δ(s,u)+1；有向圖沒有反向不等式。本例 a→b 從距離 3 指到 1，差為 2。無向邊兩方向都存在時，才有 |d[u]−d[v]|≤1。

等正權 w 時，最短權重是 w・d[v]。一般權重不可用 BFS：s→t 權 9，s→x 權 1，x→t 權 1，BFS 的 1 邊路徑權 9 大於 2 邊路徑權 2。權重全為零時可達路徑都權 0；有負權或負 cycle 更需另行定義問題。

若要算單位權圖最短路徑數，設 ways[s]=1，其餘 0。發現白 v 時照常設 d 與入列；之後對**每條**滿足 d[v]=d[u]+1 的 u→v 加上 ways[u]。同一條簡單圖邊只算一次。所有距離 k−1 的點都在距離 k 的點出列前完成，故出列 u 時 ways[u] 已收到所有最短前驅貢獻。

例如 0→1,0→2,1→3,2→3,1→4,2→4,3→5,4→5：d=(0,1,1,2,2,3)，ways=(1,1,1,2,2,4)。這是 O(n+m) 次加法；若計入大整數成本，路徑數可能有 Θ(n) 位元，不能把每次加法無條件視為固定時間。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

22.3 Depth-first search

DFS（深度優先搜尋）先完成目前遞迴分支，再處理下一鄰點。外層必須掃全部頂點，才能包含不從第一個根可達的部分。以下 time 是**發現與完成事件**的計數，不是處理邊的計數。

```
DFS(G,order):
  for v in V: color[v]=WHITE; π[v]=NIL
  time=0; finished=empty list
  for u in order:
    if color[u]==WHITE: Visit(u)
Visit(u):
  time=time+1; tin[u]=time; color[u]=GRAY
  for v in Adj[u]:
    if color[v]==WHITE: π[v]=u; Visit(v)
  color[u]=BLACK; time=time+1; tout[u]=time
  append(finished,u)
```

每個點只被 Visit 一次，每個鄰接項掃一次：Θ(n+m) 時間，遞迴深度最壞 n、額外空間 O(n)。實際寫迭代版本時，stack frame 要保存「下一個待掃鄰點」或完成事件；只把鄰點一次全 push 的 visited 走訪不自動給相同 DFS forest 或完成時間。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

發現與完成區間為何只能巢狀或分離

考慮不同頂點 u、v，設 tin[u]<tin[v]。若 v 在 u 完成前發現，當時 u 的 Visit 還在 stack 中；v 是 u 的後代，且 v 的 Visit 必先返回，故 tin[u]<tin[v]<tout[v]<tout[u]。若 v 在 u 完成後才發現，則 tin[u]<tout[u]<tin[v]<tout[v]，兩區間分離。

這排除了交錯 tin[u]<tin[v]<tout[u]<tout[v]。反過來，嚴格包含的時間區間意味發現 v 時 u 仍在 stack，故 v 是 u 的真後代。這就是 parenthesis theorem（括號定理）；它依賴遞迴必須後進先出地返回。

六點圖按 a,b,c,d,e,f 做 DFS，得到下表。括號串為 (a(b(c c)b)a)(d(e e)d)(f f)，所有數字 1 到 12 各使用一次。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

發現與完成區間為何只能巢狀或分離 （續）

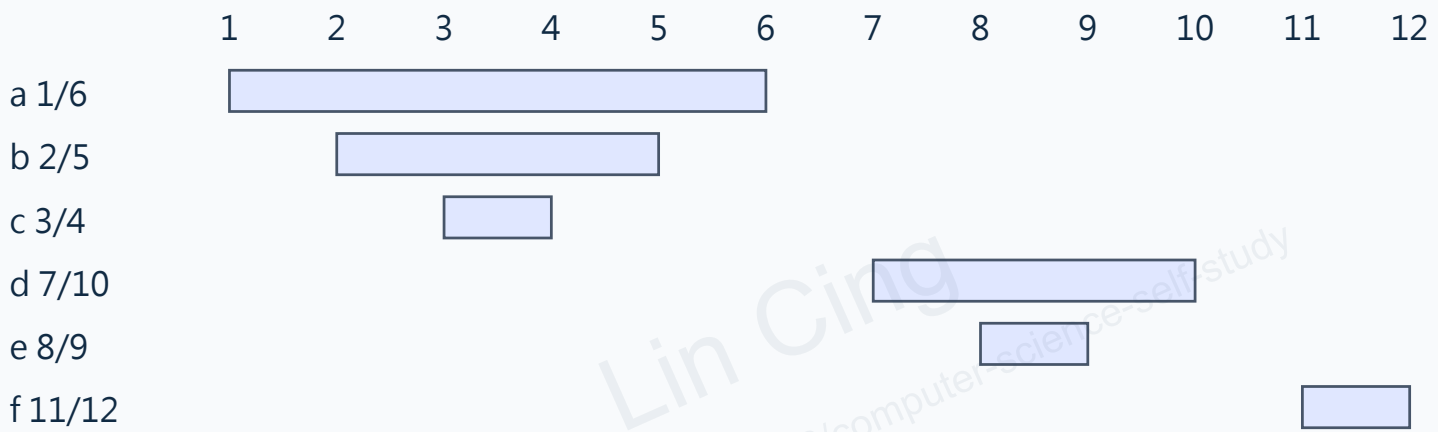
點	π	tin	tout
a	NIL	1	6
b	a	2	5
c	b	3	4
d	NIL	7	10
e	d	8	9
f	NIL	11	12

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

發現與完成區間為何只能巢狀或分離 （續）

DFS 的發現 / 完成事件

每條橫帶從 tin 到 tout；子帶巢狀，另兩棵根的區間分離。



a 包住 b,c；d 包住 e；f 是第三根。不同點區間不可能交錯。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

White-path Theorem 的時間條件

在 Visit(u) **尚未把 u 改灰之前**，v 是本次 DFS 中 u 的後代（允許 v=u），當且僅當當時有 u 到 v 的路徑，其所有點都為白。這個時間切點讓 u 本身也符合白色條件。

「後代⇒白路徑」：forest 上 u 到 v 的 tree path 是原圖的路徑；u 的各真後代都在 tin[u] 之後才發現，故當時整條 path 為白。

「白路徑⇒後代」：沿白路徑找第一個假設沒有成為 u 後代的點 y，其前點 x 已是 u 的後代（可能就是 u）。Visit(x) 掃 x→y 時，若 y 仍白就遞迴，y 必為後代；若已發現，因 y 在 tin[u] 切點仍白，它是在 u 的尚未結束遞迴期間發現，也必屬 u 的子樹。兩種情況都矛盾。

必須保留「當時全白」：圖 a→b,a→c,c→b，先掃 b。到 c 被發現時 b 已黑；雖然 c 可達 b，b 不會變成 c 的後代。一般可達性不能直接取代 white-path 條件。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

有向 DFS 的四種邊

掃描 u→v 時，u 是當下最深的灰色點，全部灰點恰為一條 stack 祖先鏈。v 白則是 tree edge，v 灰則是 back edge（回邊，含 self-loop）；v 黑時，用 tin 比較：tin[u]<tin[v] 是 forward edge，反之是 cross edge。

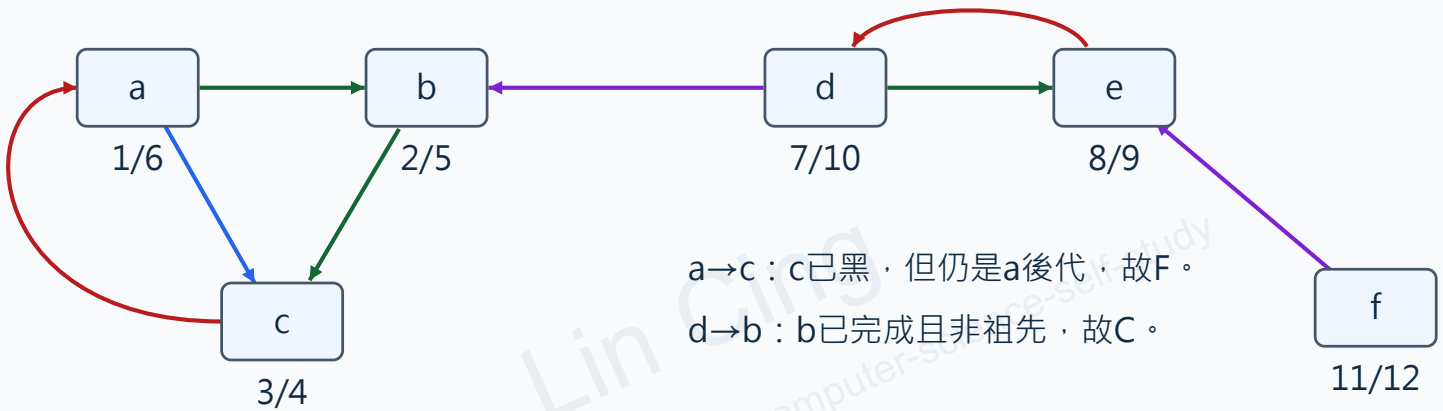
邊型	forest 關係	完成後可核對的時間式
tree	π[v]=u	tin[u]<tin[v]<tout[v]<tout[u]
forward	非 tree 的真後代邊	同上；只靠時間不能區分 tree/forward
back	v 為 u 的祖先，或 v=u	祖先時 tin[v]<tin[u]<tout[u]<tout[v]
cross	彼此無祖先關係	tout[v]<tin[u]；掃描方向不能倒過來讀

本例依實際掃描順序：a→b(T)、b→c(T)、c→a(B)、a→c(F)、d→b(C)、d→e(T)、e→d(B)、f→e(C)。a→c 是在 c 完成後才由 a 掃到，故看到黑色不代表一定是 cross。

有向 DFS 的四種邊 （續）

同一 DFS 的四種有向邊

T綠 = tree；B紅 = back；F藍 = forward；C紫 = cross。



數字為 tin/tout；圖保留8條原圖邊，箭頭方向決定分類。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

無向圖的邊型與 Cycle 判斷

無向邊會在兩個 adjacency list 出現，但作為一條邊分類時，取**第一次掃到該邊**的方向。設 tin[u] <tin[v]；因 u 的 list 中有 v，v 必在 u 完成前發現。第一次掃到時，若是 u→v，v 尚白，否則早已由 v→u 掃到，所以是 tree；若先掃 v→u，u 仍灰，是 back。故無向 DFS 只有 tree/back，沒有真正的 forward/cross。

對無向簡單圖檢查 cycle，遇到灰色父節點的反向鄰接項不能當成額外 cycle。若允許平行邊，則須用**邊識別碼**略過同一條 parent edge，不能略過所有指向 parent 的邊；第二條平行邊可形成長度 2 的 cycle（依允許 multigraph cycle 的慣例）。

有向圖則有 cycle ⇔ DFS 出現 back edge。回邊 u→v 搭配 forest 中 v 到 u 的有向 tree path 構成 cycle。反向證明：取 cycle 中最早發現的 v，當時其餘 cycle 點皆白；循 cycle 到 v 的前點 u 是白路徑，由 white-path theorem，u 是 v 的後代，所以最後一條 u→v 為 back edge。self-loop 直接落在同一結論。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

22.4 Topological sort

Topological order（拓撲序）是所有頂點的排列，使每條 u→v 都滿足 pos[u]<pos[v]。有向 cycle 要求沿圈嚴格遞增又回到原位，不可能成立；DAG（有向無環圖）才存在此排列。

在 DAG 上做 DFS，把完成順序反轉即可，不必用比較排序。對任一掃描的 u→v：v 不可能灰，否則有 back edge/cycle；v 白時成為後代，tout[v]<tout[u]；v 黑時早已完成，仍有 tout[v]<tout[u]。故按完成時間遞減，所有邊都由前指後。時間 Θ(n+m)，額外空間 O(n)。

實作可在掃邊遇灰時回報 cycle，最後才交付拓撲序。不能對一般有向圖不檢查就把 reverse-finish list 標為合法排序；完成時間總能列出，但邊限制不一定成立。

例 a→c,b→c,c→d：按 a,b,c,d 做 DFS，tin/tout 分別 a=1/6,c=2/5,d=3/4,b=7/8，反轉完成序得 b,a,c,d。a,b,c,d 也合法；**a,c,b,d 不合法，因 b→c 被逆置**。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Kahn 的入度更新與失敗判定

先算 indegree[v]，把所有入度零點放 queue。每次取 u 輸出，對每條 u→v 執行 indegree[v]−=1；當且僅當變成零時入列。計數始終等於尚未輸出頂點誘導子圖中的入度，因此輸出 u 時沒有仍未輸出的前驅，已輸出的 prefix 永遠符合邊的順序。

若剩餘非空卻無入度零點，任取殘餘點，反覆沿一條殘餘入邊走；有限頂點必重複，得到 cycle。反之 DAG 的非空誘導子圖必有入度零點，所以演算法會輸出全部 n 點。只輸出少於 n 個就回報失敗；**殘餘點可能只是 cycle 的下游，不代表每個殘餘點都在 cycle 上**。

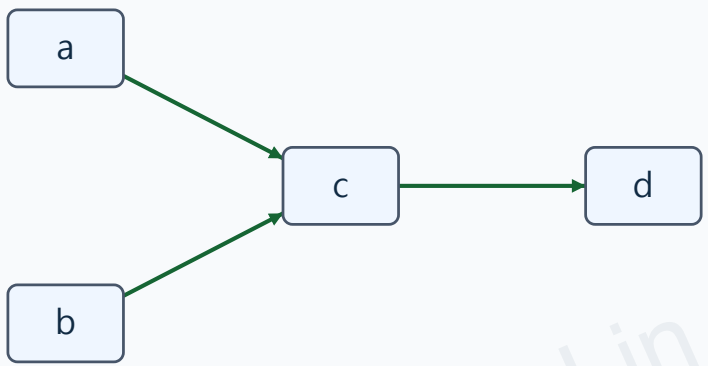
目前輸出 u	c 的剩餘入度	d 的剩餘入度	輪末 queue
初始	2	1	a,b
a	1	1	b
b	0	1	c
c	0	0	d
d	0	0	空

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Kahn 的入度更新與失敗判定 （續）

Kahn：入度與 Queue 的同步狀態

同一 DAG：a→c、b→c、c→d；移除起點後才減各出邊終點入度。



操作	in(c)	in(d)	Queue
初始	2	1	a,b
移除a	1	1	b
移除b	0	1	c
移除c	0	0	d
移除d	0	0	空

輸出 a,b,c,d；a,c,b,d 把 b→c 逆置，不合法。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

22.5 Strongly connected components

定義 u~v ⇔ u 可達 v 且 v 可達 u。零長路徑給自反性，定義給對稱性，串接路徑給遞移性，因此它是等價關係；其等價類是 SCC（強連通分量），也就是按包含關係極大的互相可達集合，**不是最大大小的那一類**。

縮合圖 condensation 的點是各 SCC，不同分量間只要有一條原圖邊就加邊。它必為 DAG：若不同類沿圈可達，圈上各類彼此可達，應合為同一等價類，矛盾。把所有邊反向會同時反轉兩方向的可達路徑，所以 G 和 G^T 有完全相同的 SCC。

六點圖的 SCC 為 A={a,b,c}、D={d,e}、F={f}，縮合邊 F→D→A。注意 G^T 有 A→D→F；從 a 隨意在 G^T 起跑可以把三個 SCC 都走到。**轉置本身不會阻止跨分量，必須配合正確根順序與已訪問標記。**

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

為何不同 SCC 的完成時間有方向

以下 tin、tout 都是**第一趟 G 的 DFS**。令 tin(C)=min tin[v]、F(C)=max tout[v]，v∈C。若不同 SCC 間有 C→D，則 F(C)>F(D)。證明分兩種首次發現先後：

1. 若 tin(C)<tin(D)，C 第一個被發現的 x 當時能沿 C 內部白路徑到邊尾，再跨到 D 並走 D 內部白路徑。由 white-path theorem，C 其餘點及 D 全成 x 的後代，所以 F(C)=tout[x]>F(D)。C 內兩點間可取完全位於 C 的路徑：若路徑經外點，該外點也與 C 互相可達，會屬 C。
2. 若 tin(D)<tin(C)，D 第一點 y 可以在全白的 D 中走到全部 D，所以 F(D)=tout[y]。D 不可能到 C，否則搭配 C→D 使兩類合併；因此 y 的整棵遞迴走不到 C，C 要等 y 完成後才可能發現，故 F(C)>F(D)。

所以 G^T 的跨分量邊 C→D 一定滿足 F(C)<F(D)。這是**分量最大完成時間**的比較；不能要求 C 中每個點都比 D 中每個點晚完成。例如 a↔b、a→c，先掃 a→b，再掃 a→c，得到 tout[b]=3、tout[c]=5、tout[a]=6：F({a,b})=6>F({c})=5，但 b 比 c 早完成。真正跨類的那條 a→c 仍有 tout[a]>tout[c]；不同 SCC 的邊不可能是 back edge。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

兩趟 DFS 的完整演算法與不跨界證明

```
SCC(G):
  first=DFS(G,所有頂點的固定順序)
  roots=reverse(first.finished)    // 第一趟的完成序，先保存
  GT=transpose(G)
  第二趟重新設所有頂點為WHITE，π=NIL
  按roots依序，遇WHITE u才在GT做Visit(u)
  每次新Visit根的整棵tree輸出為一個分量
```

歸納已輸出的每棵 tree 恰為一個完整 SCC。下一個白根 u 所在 C 有全部未處理分量中最大的 F(C)：如果 C 的最大完成點已處理，依歸納 C 整類都已處理，與 u 白矛盾。

u 可沿 C 內部路徑走到全部 C；其餘 C 仍白，故第二趟會收齊。若 G^T 有 C→D 的跨界邊，F(D)>F(C)，所以 D 必已處理為黑色，Visit 不遞迴；更不可能經此跳往另一白類。因此此 tree 既不少收也不多收，完成歸納。

兩次 DFS 各 Θ(n+m)，轉置 Θ(n+m)。反轉 append 得到的完成序為 Θ(n)，不必用 O(n log n) 排序；總時間 Θ(n+m)，新增轉置與各陣列空間 O(n+m)。

同一六點圖的兩趟結果

第一趟完成序 c,b,a,e,d,f；因此第二趟候選根序 f,d,e,a,b,c。轉置的表為 a:[c]、b:[a,d]、c:[a,b]、d:[e]、e:[d,f]、f:[]（各表按字母排列）。

候選根	G^T 的實際新發現序	遇到的跨類邊	本次輸出
f	f	無	{f}
d	d,e	e→f，f 已黑	{d,e}
e	跳過；已黑	—	不新增
a	a,c,b	b→d，d 已黑	{a,b,c}
b,c	跳過；已黑	—	不新增

F(A)=6、F(D)=10、F(F)=12，與原縮合邊 F→D→A 一致遞減。若改用第一趟 discovery 遞減，c 可能在 a 前，但這個例子未必失敗；規格正確性不能靠某次剛好成功。最小反例 x→y：第一趟 tin[x]=1,tin[y]=2，若第二趟先從 y 在 G^T 走，會沿 y→x 合併兩個本應分離的 SCC。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

同一六點圖的兩趟結果 （續）

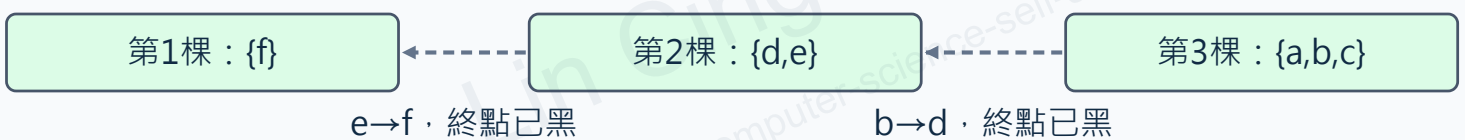
SCC：完成值排序如何阻止第二趟多收

第一趟 G 的分量最大完成值：F({f})=12、F({d,e})=10、F({a,b,c})=6。

原圖縮合 DAG



GT 的根順序與已黑邊界



綠框依序收齊一類；灰虛箭頭可跨類，但不能遞迴到已黑頂點。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

資料來源

書目

章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Chapter 22。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study